DataFrames y Spark SQL

Contents

- DataFrames y Spark SQL
- Bibliografía recomendada

Manejar RDDs resulta tedioso cuando los tipos de datos que contienen empiezan a complicarse, ya que en todo momento necesitamos saber exactamente la estructura del dato incluso cuando son tuplas jerárquicas (tuplas en las que algún campo es a su vez una tupla o lista). Sería más conveniente poder manejar los RDDs como si fuesen tablas de datos, estructuradas en filas y columnas, lo cual aportaría un mayor nivel de abstracción y más facilidad de uso. Esto es lo que nos proporciona Spark SQL y la estructura de datos fundamental, los DataFrames de Spark.

> **Note**
>
> DataFrame: tabla de datos distribuida en la RAM de los nodos, formada por filas y columnas con nombre y tipo (incluidos tipos complejos), similar a una tabla en una base de datos relacional

Internamente, un DataFrame no es más que un RDD de objetos de tipo *Row* de Spark, que representan a una fila como un vector cuyos campos tienen nombre y tipo predefinido. Cada DataFrame envuelve a un RDD de *Row* al cual podemos acceder mediante *.rdd*, por ejemplo: myDF.rdd.

A partir de la versión 2 de Spark, los creadores aconsejan utilizar exclusivamente DataFrames, y no usar los RDD sin prácticamente ninguna excepción. Aparte de la facilidad de uso, el motivo fundamental es que los DataFrames están sujetos a importantes optimizaciones automáticas de código por parte del analizador de Spark, llamado Catalyst, que **genera código al vuelo de forma automática cuando va a ejecutar el DAG con nuestras operaciones**. Catalyst optimiza el plan lógico del DAG, es decir, el orden en que se efectúan las operaciones, para obtener el mejor rendimiento, como veremos más adelante. Los RDD no están sujetos a estas optimizaciones y por tanto, la ejecución es sensiblemente más lenta

Skip to main content

Es importante tener en cuenta que los DataFrame de Spark están distribuidos en la memoria RAM de los nodos, pese a que su manejo parezca ser una tabla de una base de datos. Por otro lado, el nombre DataFrame es el mismo que el definido en otras librerías de lenguajes como Python (Dataframes del paquete Pandas) o R (data.frame). Si bien el **concepto** es el mismo (tabla de datos cuyas columnas tienen nombre y tipo), la implementación y el manejo no tienen nada que ver, más allá de que los autores eligieron el mismo nombre. Los DataFrames de Spark son un tipo de dato definido por Spark, están **distribuidos** en los nodos y se manejan mediante la API de Spark. En cambio, los Dataframes de Pandas existen solo en Python al importar específicamente esa biblioteca, no están distribuidos (toda la estructura está en la máquina única en la cual se esté ejecutando el intérprete de Python) y se utilizan mediante la API específica de Pandas. De la misma forma, la estructura data.frame de R está definida solo en el lenguaje R en el paquete base, no es distribuida y para utilizarla hay que acudir a la API de R.

En el caso de Spark, existe un método especial de los DataFrames que permiten traerse todo el contenido al programa driver como un dataframe de la biblioteca Pandas. Es el método `toPandas()`, pero debe usarse con cuidado ya que, de nuevo, requiere que todo el contenido distribuido en los nodos quepa en la memoria RAM del proceso driver. En caso contrario puede provocar una excepción *OutOfMemory*.

Operaciones con DataFrames: la API de Spark SQL

Distinguimos dos grupos de operaciones:

- Transformaciones y acciones sobre el DataFrame
- Funciones que transforman columnas

Transformaciones y acciones sobre el DataFrame

Son métodos de la clase `DataFrame` que devuelven otro DataFrame (transformaciones) o un tipo de dato primitivo. Se aplican con la sintaxis general: *resultDF = variableDF.nombreDelMétodo(argumentos)*. Es frecuente encadenar transformaciones, ya que como devuelven un DF, podemos aplicar directamente otra transformación al DF devuelvo por la transformación anterior.

La mayoría son operaciones sobre *columnas* (clase Column). Spark implementa muchas

Skip to main content

o con constantes, manipulación de columnas tipo string, comparaciones...) que debemos usar siempre que sea posible. Las operaciones con DataFrames son sometidas a optimizaciones por parte de Catalyst.

Todas ellas devuelven como resultado un nuevo DataFrame. Por eso se suelen encadenar transformaciones: *df.método1(args1).método2(args2)*

Supongamos que hemos hecho

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()
flightsDF = spark.read.csv("flights.csv", inferSchema=True, header=True).cache()
```

Ahora podemos hacer:

- Transformación `select`: Recibe los nombres de las columnas que queremos seleccionar y devuelve otro DF que solo contiene esas columnas. Todas las columnas deben existir.

  Se pueden utilizar nombres de columna como string, o utilizar *objetos Column* como argumentos de `select`. Para crear un objeto Column utilizamos la función `col` del sub-paquete `pyspark.sql.functions`. Como dicho subpaquete contiene algunas funciones que colisionan con funciones del paquete base de Python como `round`, `max`, `min`, es muy frecuente importar el subpaquete con el alias `F` y utilizar sus funciones como `F.nombrefuncion`. Las funciones de dicho paquete operan con un objeto columna y devuelven otro objeto columna, y son todas operaciones distribuidas, que se efectúan en los executors, y por tanto, sobre las filas de cada partición que exista en ese executor (es decir, cada executor aplica la operación a los trozos de columna que él tiene).

  La clase Column **re-implementa** los operadores aritméticos `+`, `-`, `*`, `/` y por tanto, el producto de un objeto columna por otro objeto column, o el producto de una columna por un número real, se llevan a cabo de manera distribuida utilizando el operador producto de la clase Column de pyspark, y no el operador de multiplicación habitual de Python, ya que éste último no sabe cómo multiplicar entre sí dos objetos Column o un Column con un número real. A esto se le llama *aritmética de columnas*.

```
from pyspark.sql import functions as F

transformed_df = flightsDF.select("Month", "DayofMonth", "ArrTime", "DepDelay")

nuevo_df = flightsDF.select(
    F.col("Month"),
    F.col("Distance"),
```

Skip to main content

```
    (1.6 * F.col("Distance")).alias("DistKm") # creamos nueva columna
)
```

- Transformación `sample:` recibe el porcentaje de filas que queremos mantener y devuelve otro DF que tiene solo ese porcentaje de forma aleatoria (el porcentaje **no es exacto sino aproximado**).

```
random_sample_df = flightsDF.sample(0.1)  # aproximadamente un 10% aleatorio de las
```

> **Note**
>
> A veces necesitamos unas pocas filas que reflejen el comportamiento general, con lo cual obtenemos una muestra aleatoria. El objetivo suele ser operar en el driver para realizar, por ejemlpo, **representaciones gráficas** con Python (no Spark) con paquetes de visualización como Seaborn, Matplotlib. El DF de Spark completo probablemente no cabría en memoria.

- Transformación `filter` / `where` (equivalentes): devuelve un nuevo DF que contiene solo las filas que cumplen cierta condición booleana definida sobre una o más columnas. Las funciones `filter()` y `where` son exactamente equivalentes. El argumento puede ser un string con una condición en SQL puro, o también una condición booleana utilizando la API de SparkSQL API sobre objetos columna, siempre que el resultado de la expresión sea un objeto columna de booleanos.

```
transformed_df = (
    # Como rodeamos estas líneas entre paréntesis, no necesitamos la barra \ al fir
    # y además podemos usar comentarios en cada línea al no haber \
    flightsDF
        .filter("DayOfMonth < 20 or Origin = 'LAX'")    # SQL puro
        .filter("CarrierDelay is not null")             # SQL puro
        .filter(~(F.col("Distance").isNull()))          # operaciones con objetos C
        .filter(1.6 * F.col("Distance") > 5000)         # operaciones con objetos C
        .filter(F.col("ArrDelay") > F.col("DepDelay") + F.col("CarrierDelay"))
        .where("ArrDelay > 15")                         # SQL puro
)

# Recordemos que en esta celda no se lleva a cabo ningún cálculo porque
# no hemos invocado ninguna acción sobre el DF resultante transformed_df
```

- Transformación `withColumn`: Devuelve un nuevo DF en el que o bien se ha creado una nueva columna o bien se ha reemplazado una columna existente, en ambos casos a partir de otras existentes. El DF devuelto contiene **todas las columnas ya existentes**

Skip to main content

Para crear la nueva columna se puede utilizar combinado con cualquiera de las funciones del paquete *pyspark.sql.functions* que veremos en el siguiente apartado, puesto que todas son ya funciones distribuidas y paralelas. Si el nombre corresponde a una columna **ya existente**, estaremos **reemplazando** (*creándola de nuevo*) en el nuevo DF en su misma posición.

```
transformed_df = (flightsDF
        .withColumn("DistKm", 1.6 * F.col("Distance")) # crear nueva columna
        .withColumn("TotalDelay", F.col("DepDelay") + F.col("ArrDelay"))
        .withColumn("Year", F.lit(2018)) # reemplazar columna existente por una col
)
```

Se suelen encadenar múltiples operaciones *withColumn* una tras otra. Las nuevas columnas se van creando por la derecha en el mismo orden en el que vamos encadenando las operaciones

- Transformación `drop`. Devuelve un nuevo DF en el que se ha(n) eliminado la(s) columna(s) indicada(s), pero no existe, no da ningún error.
- Transformación `withColumnRenamed`. Devuelve un nuevo DF en el que se ha renombrado una columna, en la misma posición que ocupaba. En este caso la columna que se está renombrando sí debe existir, o Spark dará un error. Ambas transformaciones pueden encadenarse al igual que las anteriores:

```
transformed_df = flightsDF\
    .withColumnRenamed("ArrDelay", "MinutesLate")\
    .drop("CarrierDelay", "DepDelay", "inventedName")  # esto es correcto (no obter
```

- Transformación `summary`. Devuelve un nuevo DF con un **un resumen estadístico** de las columnas numéricas. Media, desv. típica, min, max y cuartiles (Q1, Q2, Q3).
- Transformación `distinct` Devuelve un nuevo DF en el que se han eliminado las filas duplicadas, es decir, aquellas que coinciden en todos los campos. uniqueDF =

```
flights_sin_repetidos = flightsDF.distinct()

# distinct() también sirve para contar el número de valores distintos que tiene una
# columna categórica:

n_origenes = flightsDF.select("Origin").distinct().count()
```

- Transformación `dropDuplicates`. Si queremos indicar a Spark que considere la fila

Skip to main content

```
one_row_per_route_df = flightsDF.dropDuplicates(["Origin", "Dest"]) # Basta que coi
```

- Transformaciones `union`, `intersect` y `exceptAll`: devuelven un nuevo DF que es el resultado de la unión (o de la intersección, o de la diferencia, respectivamente) de las filas del DF sobre el que se aplica y el DF que se le pasa como argumento:

```
otro_df = flightsDF.where("Distance > 10000")

union_df = flightsDF.union(otro_df)
intersection_df = flightsDF.intersect(otro_df)
difference_df = flightsDF.exceptAll(otro_df)
```

- Transformación `sort / orderBy` (equivalentes). Devuelven un nuevo DF que es el resultado de ordenar las filas del original en base a una o varias columnas que le indiquemos. Cuando se indican varias columnas, se ordena en base a la primera, y en caso de empate, se desempata en base a la segunda columna, y así sucesivamente.

  Como en cualquier transformación, el DF original jamás se modifica. El resultado es un nuevo DF que está ordenado y que, a pesar de que no contiene números de fila, nos garantiza que las acciones que efectuemos sobre él y que actúan sobre las "n" primeras filas (tales como `show`, `take`, `collect`), actuarán sobre las filas en el orden en el que han quedado como resultado de la ordenación.

```
sorted_df = flightsDF.sort("DepTime")  # orden ascendente por defecto
sorted_df = flightsDF.orderBy("Origin", "ArrDelay", ascending = False)  # orden des
sorted_df = flightsDF.sort(F.col("Origin"), F.col("DepTime").desc())  # ordenar asc
                                                                      # desempatar c
```

- Acción `collect`: devuelven al driver el contenido completo del DataFrame como una lista de Python de objetos *Row*. Debe usarse con cuidado porque si es superior al tamaño de memoria del driver, causará una excepción *OutOfMemory*. Como norma general no debería utilizarse en procesos en producción.
- Acción `head`: devuelve al driver la *primera* fila del DataFrame como objeto *Row*. El concepto de *primera* solo está garantizado en caso de que el DF esté ordenado, es decir, que sea el DF devuelto por la transformación *sort /orderBy * sobre otro DF previo. Si no, simplemente devuelve una fila, sin garantizar cuál será.
- Acción `take(n)`: devuelve al driver una lista de Python de objetos `Row` con las *n* primeras fila del DataFrame, siendo *n* un argumento de tipo entero que debemos pasarle. Mismo razonamiento explicado antes acerca de la ordenación.

Skip to main content

```
listaRows = flightsDF.limit(100000).collect()   # ponemos limit en el ejemplo para

print(f"La longitud de la lista devuelta es de {len(listaRows)} elementos")
print("--------")

algunasRows = flightsDF.take(3)
firstFlight = flightsDF.head()

print(firstFlight)
print("----------")
print(f"El valor del atributo Origin es {firstFlight.Origin}")  # podemos acceder a
```

```
La longitud de la lista devuelta es de 100000 elementos
--------
Row(Month=1, DayofMonth=14, DayOfWeek=7, FlightDate=datetime.date(2018, 1, 14), Ori
----------
El valor del atributo Origin es SYR
```

- Acción `show(n=20)`: envía al driver las *n* primeras filas (20 si no indicamos nada) para que éste las imprima por pantalla (mismo razonamiento acerca de la ordenación). **La función devuelve `None` como resultado**, es decir, las filas se imprimen pero no se devuelven como resultado de la función, ya que esta devuelve el objeto nulo de Python.
- Acción `count()`: envía al driver un entero con el número total de filas del DF sobre el que la aplicamos.

Funciones que transforman columnas

Son funciones pertenecientes al paquete `pyspark.sql.functions` (existen 420 funciones en dicho subpaquete, según la documentación de pyspark versión 3.5.0) y todas reciben como entrada uno o varios objetos Column de un DataFrame, y devuelven generalmente otro objeto Column transformado. Todas las operaciones están implementadas de forma distribuida, por lo que cada executor las aplica sobre las filas que posee. No se aplican al objeto DataFrame sino que se invocan directamente como funciones. Es frecuente importar el paquete *functions* completo con algún alias, por ejemplo F, para no interferir esas funciones con respecto a otras que se llamen igual en Python (por ejemplo min, max), y usarlas siempre con ese prefijo: F.col(...), F.when(...), etc. La mayoría de estas operaciones reciben como argumento un string con el nombre de una o varias columnas a las que se aplica, aunque algunas requieren pasarles el objeto columna con F.col("nombre") en vez de directamente el string "nombre".

Existe una gran variedad de funciones para transformar columnas de un DataFrame. Solo

Skip to main content

- Función `F.col("nombreCol")`: sirve simplemente para obtener un objeto Column a partir del nombre de la columna. Se suele utilizar dentro de otras funciones para referenciar a columnas de un DataFrame con las que queremos operar.
- Función `F.lit(constante)`: sirve para crear una columna constante, es decir, con todas sus filas igual al valor que le pasemos como argumento, que puede ser un número, un string, un booleano, ...
- Función `F.when`: devuelve un nuevo objeto columna en base a casuísticas:
  `F.when(cond1, valor1).when(cond2, valor2).otherwise(valor3)`

  Los valores *valor1, valor2, valor3* pueden ser constantes con F.lit(...) o pueden ser objetos Column, es decir, objetos columna existentes u operaciones con ellos. Se utiliza generalmente dentro de la transformación *withColumn*. La creación del nuevo objeto columna se lleva a cabo en los executors, fila a fila, de manera que en cada fila se comprobará la condición y se elegirá el valor que debe colocarse en esa fila de la nueva columna. Dicho valor puede ser una constante o puede ser el valor que ya hubiera en otra columna de esa misma fila.

```
flights_no_negativos_df = flightsDF.withColumn("ArrDelay",
    F.when(F.col("ArrDelay") < 0, F.lit(0))  # Modificamos el valor si era negativo
    .otherwise(F.col("ArrDelay"))            # Tomará el valor que hubiera en esta
)
```

El valor devuelto por *when* es un objeto Column, que es lo que le pasamos a *withColumn* como segundo argumento (el primero es el nombre de la nueva columna).

- Funciones matemáticas y estadísticas que devuelven objetos columna:
  - Columna de números aleatorios: `F.rand` (uniforme en [0,1]) o `F.randn` (normal estándar). Ejemplo: `df.withColumn("aleat", F.rand())`
  - Correlaciones entre columnas (devuelve un número): `F.corr("col1", "col")`, tablas de contigencia.
  - Funciones de aritmética entre columnas: `F.col("ArrDelay") + F.col("DepDelay")` devuelve otro objeto columna con la suma.
  - Operaciones matemáticas sobre una columna: `F.sqrt("colName")`, `F.sin("colName")`, `F.log("colName")`...
  - Suelen utilizarse también dentro de *withColumn* aunque pueden usarse también dentro de *select* para crear al vuelo una nueva columna.

Skip to main content

Métodos de la clase Column

La clase Column posee varios métodos pero hay dos muy frecuentes. Transforman el objeto Columna al que se aplican para **renombrarlo** (**alias**, usado en columnas creadas al vuelo) o para **cambiar su tipo de dato (cast)**. Importante: el tipo indicado debe ser un *tipo de dato de Spark* de los existentes en `pyspark.sql.types`. El tipo de dato se puede indicar con un string, como `"int", "double", "boolean"` o como `IntegerType()`, `BooleanType()`, etc.

```
from pyspark.sql.types import IntegerType

flightsTransformadoDF = flightsDF.select(
    F.col("ArrDelay").cast(IntegerType()),   # convertimos a entero
    (1.6 * F.col("Distance")).alias("DistKm")    # damos nombre a la nueva col
)
```

Ejemplos con funciones matemáticas y estadísticas

Generar con select dos columnas de distribuciones uniforme y normal

```
flightsDF.select(
    F.rand(seed=10).alias("uniforme"),  # aleatorio siguiendo una distribución unif
    F.randn(seed=27).alias("normal")   # aleatorio siguiendo una Normal estándar (m
).show()
```

```
+------------------+--------------------+
|          uniforme|              normal|
+------------------+--------------------+
| 0.1709497137955568| -0.8664700627108758|
| 0.8051143958005459| -0.5970491018333267|
| 0.5775925576589018| 0.18267161219540898|
| 0.9476047869880925| -1.8497305679917546|
|    0.2093704977577|  0.9410417279045351|
|0.36664222617947817| -0.6516475674670159|
| 0.8078688178371882|  0.5901002135239671|
| 0.7135143433452461|  -1.850241871360443|
| 0.7195325566306053| 0.09176896733073023|
|0.31335292311175456|-0.38605118617831075|
| 0.8062503712025726|  1.2134544166783332|
|0.10814914646176654| -1.0757702531630617|
| 0.3362232980701172| 0.04961226872064977|
| 0.8133304803837667|  -0.768259602441542|
|0.47649428738170896|  0.2911293146907403|
|  0.524728096293865|-0.33406080411047484|
| 0.9701253460019921|  1.3607097640771781|
| 0.6232167713919952|  0.5986772981082732|
| 0.5089687568245219|-0.35158579838711623|
```

Skip to main content

```
+-------------------+--------------------+
only showing top 20 rows
```

Podemos usar *describe* para calcular estadísticos descriptivos simples de cada columna que le pasemos como argumento, o de todas las columnas si no le pasamos ninguna

```
df = flightsDF.limit(1000)\
              .withColumn("rand1", F.rand(seed=10))\
              .withColumn("rand2", F.rand(seed=27))

cov = df.stat.cov("rand1", "rand2")
corr = df.stat.corr("rand1", "rand2")

print(f"La covarianza es {cov} y la correlación lineal de Pearson es {corr}")
```

```
La covarianza es -0.0002591695400945357 y la correlación lineal de Pearson es -0.00
```

Tablas de contingencia

Creamos un DataFrame con dos columnas (name, item)

```
names = ["Alice", "Bob", "Mike"]

items = ["milk", "bread", "butter", "apples", "oranges"]

df = spark.createDataFrame([(names[i % 3], items[i % 5]) for i in range(100)],
                           ["name", "item"]).cache()

# Vamos a mostra 10 filas por pantalla

df.show(10)

# Lo siguiente equivale a df.groupBy("name").pivot("item").count()
df.stat.crosstab("name", "item").show()
```

```
+-----+-------+
| name|   item|
+-----+-------+
|Alice|   milk|
|  Bob|  bread|
| Mike| butter|
|Alice| apples|
|  Bob|oranges|
| Mike|   milk|
|Alice|  bread|
|  Bob| butter|
```

Skip to main content

```
+-----+-------+
only showing top 10 rows
```

```
+---------+------+-----+------+----+-------+
|name_item|apples|bread|butter|milk|oranges|
+---------+------+-----+------+----+-------+
|      Bob|     6|    7|     7|   6|      7|
|    Alice|     7|    7|     6|   7|      7|
|     Mike|     7|    6|     7|   7|      6|
+---------+------+-----+------+----+-------+
```

Operaciones con columnas de tipo vector

Funciones que se aplican a un objeto Column de tipo array y devuelven un nuevo objeto Column donde la función se ha aplicado a cada celda de la columna original. Existen funciones que devuelven el elemento de alguna posición concreta de cada vector, o bien la longitud de cada vector, el mínimo, el máximo, el propio vector ordenado, el array con los elementos comunes a dos columnas de tipo vector, su unión, el subvector de elementos que cumplen una condición (filtrado del vector), la explosión entendida como desenrollar los elementos repitiendo las filas para que cada elemento del vector se asocie a una fila completa diferente (manteniendo constante los elementos de las demás columnas), ... Podemos consultar en la documentación ya que la mayoría empiezan por "array_". La tabla de la página siguiente contiene un resumen de algunas.

Ejemplos:

```
todas_fechas_df = flightsDF\
    .groupBy("Origin", "Dest")\
    .agg(F.collect_list("FlightDate").alias("todas_fechas"),
         F.collect_set("FlightDate").alias("fechas_sin_repetidos"))\
    .withColumn("longitud_todas_fechas", F.size("todas_fechas"))\
    .withColumn("longitud_fechas_sin_rep", F.size("fechas_sin_repetidos"))\
    .withColumn("primer_elemento", F.element_at("todas_fechas", 1))   # primer elem

todas_fechas_df.select("Origin", "Dest", "todas_fechas", "longitud_todas_fechas", '
               .show(3)
```

```
+------+----+--------------------+--------------------+--------------------+------
|Origin|Dest|        todas_fechas|longitud_todas_fechas|fechas_sin_repetidos|longit
+------+----+--------------------+--------------------+--------------------+------
|   ABE| CLT|[2018-01-01, 2018...|                 264|[2018-01-15, 2018...|
|   ABE| FLL|[2018-01-03, 2018...|                  34|[2018-03-07, 2018...|
|   ABI| DFW|[2018-01-01, 2018...|                 629|[2018-01-15, 2018...|
```

Skip to main content

```
+------+----+--------------------+--------------------+--------------------+------
only showing top 3 rows
```

Skip to main content

Nombre de la función que se aplica a columnas de tipo vector	Descripción
array_contains(column: Column, value: Any)	Comprueba si un valor se presenta en una columna tipo vector. Devuelve los valores siguientes. True: devuelve si el valor existe en el vector. False: cuando el valor no está presente. None: cuando el vector es nulo.
array_distinct(e: Column)	Devuelve una columna de tipo vector tras eliminar duplicados de cada celda.
array_except(col1: Column, col2: Column)	Devuelve todos los elementos que están en el vector col1 pero no en col2
array_intersect(col1: Column, col2: Column)	Devuelve todos los elementos que están presentes en los vectores col1 y col2.
array_join(column: Column, delimiter: String, nullReplacement: String)	Concatena entre sí todos los elementos del vector utilizando el delimitador proporcionado. Cuando hay valores nulos, se reemplazan con la cadena 'nullReplacement'
array_join(column: Column, delimiter: String)	
array_max(e: Column)	Devuelve el valor máximo del vector
array_min(e: Column)	Devuelve el valor mínimo del vector
array_position(column: Column, value: Any)	Devuelve la posición/índice de la primera aparición del "valor" en el vector (empezando en 1). Devuelve cero cuando no se encuentra el valor. Devuelve nulo cuando alguno de los argumentos es nulo.
array_remove(column: Column, element: Any)	Devuelve una columna tipo vector tras eliminar el elemento del segundo argumento.
array_repeat(e: Column, count: Int)	Crea un vector que contiene el primer argumento repetido el número de veces indicado por el segundo argumento.

Skip to main content

Nombre de la función que se aplica a columnas de tipo vector	Descripción
array_repeat(left: Column, right: Column)	Crea un vector que contiene el primer argumento repetido el número de veces indicado por la columna del segundo argumento.
array_sort(e: Column)	Devuelve una versión ordenada del vector. Los valores nulos se colocan al final
array_union(col1: Column, col2: Column)	Devuelve un vector de elementos que están presentes en ambos vectores (unión de vectores)
arrays_overlap(a1: Column, a2: Column)	True: si `a1` y `a2` tienen al menos un elemento no nulo en común. False: si `a1` y `a2` tienen elementos completamente diferentes. None: si ambas matrices no están vacías y cualquiera de ellas contiene un “nulo”
arrays_zip(e: Column*)	Devuelve una columna vector de tuplas de dimensión igual al nº de argumentos, en las que la n-ésima tupla contiene todos los n-ésimos valores de cada una de las columnas
concat(exprs: Column*)	Concatena todos los elementos de una columna determinada.
element_at(column: Column, value: Any)	Devuelve el elemento del vector situado en la posición de entrada “valor”.
exists(column: Column, f: Column => Column)	Comprueba si el vector contiene un elemento que cumpla la condición de la función
explode(e: Column)	Crea una fila para cada elemento en la columna del vector
explode_outer (e : Column)	Crea una fila para cada elemento en la columna del vector. A diferencia de la explosión, si el vector es nulo o está vacío, devuelve nulo.
filter(column: Column, f: Column => Column)	Devuelve un vector de elementos para los que se cumple un predicado (func. booleana)

Skip to main content

Nombre de la función que se aplica a columnas de tipo vector	Descripción
filter(column: Column, f: (Column, Column) => Column)	
flatten(e: Column)	Crea un vector a partir de una columna de vector de vectores (aplanar).
forall(column: Column, f: Column => Column)	Devuelve True si un predicado se cumple para cada elemento del vector, False si no.
posexplode(e: Column)	Crea una fila para cada elemento del vector y crea dos columnas “pos” para mantener la posición del elemento en el vector y la “col” para mantener el valor real del elemento.
posexplode_outer(e: Column)	Ídem al anterior, pero a diferencia de posexplode, si el vector es nulo o está vacío, devuelve (nulo, nulo) para las columnas pos y col.
reverse(e: Column)	Devuelve el vector de elementos en orden inverso.
shuffle(e: Column)	Desordena aleatoriamente los elementos del vector (“barajar” los elementos).
size(e: Column)	Devuelve la longitud de un vector.
slice(x: Column, start: Int, length: Int)	Devuelve el vector de elementos empezando en la posición “inicio” y la longitud dada.
sort_array(e: Column)	Ordena el vector en orden ascendente. Los valores nulos se colocan al principio.
sort_array(e: Column, asc: Boolean)	Ordena el vector en orden ascendente o descendente según el parámetro booleano. Ascendentemente: los valores nulos se colocan al principio. En caso contrario, al final
transform(column: Column, f: Column => Column)	Devuelve un vector de elementos después de aplicar a cada elemento la función de transformación. Si la

Skip to main content

Nombre de la función que se aplica a columnas de tipo vector	Descripción
	el elemento del vector y la posición que ocupa dicho elemento.
transform(column: Column, f: (Column, Column) => Column)	

Columnas de tipo estructura (tupla)

Una columna de tipo estructura representa un tipo de dato creado por el usuario, que puede ser “plano”, es decir, ser simplemente una tupla, o puede ser “jerárquico” porque dentro de la estructura existan campos que son, a su vez, otras estructuras. El caso más simple es crear una tupla con los valores de varias columnas, fila a fila. Es posible “ordenar” tuplas, porque ordena en base al primer elemento de cada tupla (el más a la izquierda), y si empatan entonces se fija en el segundo, y así sucesivamente. Esto es útil cuando queremos ordenar dentro de un grupo en base a cierta columna, y después queremos ver el valor concreto de otra columna diferente que estaba en la misma fila, emparejado con el valor máximo o mínimo del grupo (es decir, cuando buscamos máximo o mínimo, pero no queremos perder la correspondencia de ese valor con otros valores de su fila).

Se puede acceder a cada uno de los campos de una tupla con el operador . (punto)

Ejemplo: para cada par de aeropuertos origen y destino, encontrar la fecha en la que tuvo lugar el vuelo con más retraso a la llegada. Como estamos creando una columna de tipo estructura con dos campos, de los cuales el primero es el ArrDelay, al buscar dentro de cada grupo el máximo de dicha columna de tipo estructura, se fijará justamente en el ArrDelay.

```
from pyspark.sql import Window

fechaMaxDelayDF = flightsDF\
                  .withColumn("estructura", F.struct("ArrDelay", "FlightDate"))\
                  .groupBy("Origin", "Dest")\
                  .agg(F.max("estructura").alias("max_struct"))\
                  .withColumn("fecha_max_retraso", F.col("max_struct.FlightDate")

fechaMaxDelayDF.show(5, truncate=False)
```

Skip to main content

```
+------+----+------------------+----------------+
|ABE   |ATL |{459.0, 2018-02-20}|2018-02-20      |
|ABE   |CLT |{347.0, 2018-02-04}|2018-02-04      |
|ABE   |DTW |{674.0, 2018-02-09}|2018-02-09      |
|ABE   |FLL |{340.0, 2018-03-07}|2018-03-07      |
|ABE   |MDT |{NULL, 2018-01-12} |2018-01-12      |
+------+----+------------------+----------------+
only showing top 5 rows
```

Transformaciones para agrupar y agregar

La transformación *groupBy("nombreCol1", "nombreCol2", ...)* sobre un DataFrame devuelve una estructura de datos llamada *RelationalGroupedDataset*, que **no** es un DataFrame y sobre la que apenas se pueden aplicar operaciones. Las más frecuentes que se suelen aplicar son *count*(), que efectúa un conteo del número de elementos por cada grupo, o la función *agg*() que es la más habitual, y realiza para cada grupo las agregaciones que le indiquemos en los argumentos de agg(). El resultado tendrá tantas filas como combinaciones de valores posibles haya en las variables de grupo, y solamente contendrá aquellas columnas que se incluyeron como argumentos en el *groupBy* (que definen los grupos) más una columna por cada función de agregación que incluimos como argumento de *agg()*. La función *pivot*() también sirve para agupar, pero desplegando como columnas cada una de las combinaciones de categorías de las variables que le pasamos como argumento.

```
import pyspark.sql.functions as F

summaries_df = flightsDF\
    .groupBy("Origin", "Dest")\
    .agg(
         F.max("Distance").alias("maxDistance"),
         F.mean("ArrDelay").alias("meanArrDelay"),
         F.mean("DepDelay").alias("meanDepDelay"),
         F.stddev("ArrDelay").alias("stddevArrDelay"),
         F.count("*").alias("groupSize"),
         F.countDistinct("Dest").alias("distinctDestinations")
    )

summaries_df.show()
```

```
+------+----+-----------+------------------+------------------+------------------
|Origin|Dest|maxDistance|       meanArrDelay|       meanDepDelay|    stddevArrDelay
+------+----+-----------+------------------+------------------+------------------
|   BMI| ATL|      533.0| 17.115727002967358|  20.11111111111111| 74.55372157534977
|   FLL| LGB|     2327.0| 18.266666666666666|             28.575|59.455898963590045
|   FLL| CVG|      932.0|  5.612903225806452| 11.330645161290322|44.525334584674795
|   BIS| AZA|     1092.0|  7.526315789473684| 11.355263157894736|  59.3011464047052
|   ATL| FLL|      581.0|-0.0657752078062884|  6.979429808733309|25.828695002011504
```

Skip to main content

```
|   DTW| LEX|      296.0|  2.216589861751152|  9.554022988505746|55.302352654559996
|   LIT| DEN|      771.0| 10.050314465408805| 11.081761006289309|57.802808347844525
|   BUF| PHX|     1912.0|  11.98360655737705| 19.360655737704917| 35.51067435992755
|   DSM| STL|      259.0|-0.2850678733031674| 3.3755656108597285|19.548429456094873
|   GEG| OAK|      723.0| -4.531120331950207|  2.954356846473029| 19.69517883026943
|   DEN| LGB|      854.0|-3.0294117647058822| 4.0588235294117645|15.196660905402767
|   SIT| JNU|       95.0|-1.2258064516129032|-2.4566210045662102|36.296868518884395
|   MSP| BIS|      386.0|  4.032786885245901|  9.153970826580228| 41.25493142164648
|   CVG| CLT|      335.0| 11.320866141732283| 13.566666666666666| 45.82282647594028
|   SJC| JFK|     2569.0|  4.731182795698925|  20.91578947368421| 30.54367499633009
|   PIE| CID|     1090.0| 2.5714285714285716|   6.73469387755102| 50.41411839289995
|   CVG| LAX|     1900.0|             11.692|  8.466135458167331| 34.99508192640995
|   PIE| GSO|      587.0|-3.1714285714285713| 6.0285714285714285|34.055737803062264
+------+----+-----------+-------------------+-------------------+------------------
only showing top 20 rows
```

Cuando ejecutamos funciones de agregación sobre el DF completo, sin haber llevado previamente una agrupación con *groupBy*, se devuelve un DataFrame con una sola fila con el resultado de la agregación. El fragmento de código siguiente muestra cómo se utilizan *groupBy* y *agg*.

Agregación con funciones de ventana

Son similares a una agregación por grupos, pero se usan para añadir la agregación en una nueva columna del DF original. **No modifican el número de filas del DF original** sino que calculan la agregación en cada grupo y la añaden en las filas que pertenecen a ese grupo. Suele utilizarse dentro de *withColumn*. Los grupos se definen en el objeto *Window* que hay que crear y que se utiliza para agregar. A continuación comparamos el comportamiento de groupBy y el de una agregación con ventana.

En el ejemplo siguiente, vemos que a cada vuelo se le ha "pegado" el retraso medio del grupo al cual pertenece, es decir, el retraso medio de los vuelos que tienen su mismo aeropuerto de origen. Por eso, vuelos que tienen mismo origen aunque tengan distinto destino muestran el mismo valor en `MeanDelay`.

```
from pyspark.sql import Window

w = Window().partitionBy("Origin")

# tantas filas como aeropuertos de origen \# tantas filas como tenga flightsDF
flightsDF.withColumn("MeanDelay", F.mean("ArrDelay").over(w))\
         .select("Month", "DayofMonth", "DayOfWeek", "Origin", "Dest", "ArrDelay",
         .dropDuplicates(["Origin", "Dest"])\
         .show()
```

Skip to main content

```
+-----+----------+---------+------+----+--------+-----------------+
|Month|DayofMonth|DayOfWeek|Origin|Dest|ArrDelay|        MeanDelay|
+-----+----------+---------+------+----+--------+-----------------+
|    1|         2|        2|   BGM| DTW|    -2.0|16.874538745387454|
|    1|         2|        2|   BTM| SLC|     0.0| 3.009009009009009|
|    1|         1|        1|   BUR| JFK|    -2.0|4.9543734953571015|
|    1|         2|        2|   BUR| SFO|    33.0|4.9543734953571015|
|    1|         2|        2|   BUR| DEN|     1.0|4.9543734953571015|
|    1|         3|        3|   BUR| SJC|   -18.0|4.9543734953571015|
|    1|         3|        3|   BUR| SLC|   -28.0|4.9543734953571015|
|    1|         3|        3|   BUR| DAL|   -13.0|4.9543734953571015|
|    1|         3|        3|   BUR| LAS|    -6.0|4.9543734953571015|
|    1|         3|        3|   BUR| OAK|   -20.0|4.9543734953571015|
|    1|         3|        3|   BUR| PDX|   -12.0|4.9543734953571015|
|    1|         3|        3|   BUR| PHX|    10.0|4.9543734953571015|
|    1|         3|        3|   BUR| SMF|   -13.0|4.9543734953571015|
|    1|         2|        2|   BUR| SEA|     3.0|4.9543734953571015|
|    1|         2|        2|   COD| SLC|     3.0|3.2696245733788394|
|    3|         4|        7|   COD| DEN|   245.0|3.2696245733788394|
|    1|         1|        1|   DCA| BOS|   -21.0|  4.12925289757857|
|    1|         1|        1|   DCA| PBI|     5.0|  4.12925289757857|
|    1|         1|        1|   DCA| RSW|    -4.0|  4.12925289757857|
|    1|         1|        1|   DCA| TPA|    34.0|  4.12925289757857|
+-----+----------+---------+------+----+--------+-----------------+
only showing top 20 rows
```

En un objeto Window pueden definirse:

- Grupos, mediante `partitionBy("col1", "col2")`. Las funciones de agregación se calcularán para cada grupo de la ventana, y el valor del grupo adecuado (en base a los valores que toman en esa fila las columnas que definen el grupo) será añadido en la nueva columna que hemos creado con *withColumn*. Las funciones de agregación que se suelen aplicar son las mismas que con *groupBy*.
- Ordenaciones, mediante `orderBy*("col1", "col2")`. Sirven para casos en los que queremos aplicar a cada fila una función de agregación que depende del orden, como por ejemplo
  - `F.first("col1")` o `F.last("col1")`, que devuelven el menor o el mayor valor de cierta columna por la que previamente hemos ordenado la ventana.
  - La función `F.rank()`, que crea una columna de enteros con la *posición* que ocupa cada fila dentro de un grupo que hemos ordenado previamente.
  - En una ventana ordenada también se puede aplicar la función `F.lag("colName")` para calcular la diferencia entre el valor de "colName" de una fila y el de la fila siguiente en la ordenación, o el de *count* filas posteriores si le pasamos un segundo argumento entero. Se usa con frecuencia en problemas de series temporales.

Ejemplo:

Skip to main content

```
from pyspark.sql import Window

w = Window().partitionBy("Origin").orderBy("FlightDate")

flightsDF.withColumn("MeanDelay", F.mean("ArrDelay").over(w))\
         .withColumn("rankByDate", F.rank().over(w))\
         .select("Month", "DayofMonth", "FlightDate", "Origin", "Dest", "MeanDelay"
         .show()
```

```
+-----+----------+----------+------+----+------------------+----------+
|Month|DayofMonth|FlightDate|Origin|Dest|         MeanDelay|rankByDate|
+-----+----------+----------+------+----+------------------+----------+
|    1|         1|2018-01-01|   BGM| DTW|              -3.0|         1|
|    1|         2|2018-01-02|   BGM| DTW|             -8.75|         2|
|    1|         2|2018-01-02|   BGM| DTW|             -8.75|         2|
|    1|         2|2018-01-02|   BGM| DTW|             -8.75|         2|
|    1|         3|2018-01-03|   BGM| DTW|-4.833333333333333|         5|
|    1|         3|2018-01-03|   BGM| DTW|-4.833333333333333|         5|
|    1|         4|2018-01-04|   BGM| DTW|            -1.375|         7|
|    1|         4|2018-01-04|   BGM| DTW|            -1.375|         7|
|    1|         5|2018-01-05|   BGM| DTW|               2.2|         9|
|    1|         5|2018-01-05|   BGM| DTW|               2.2|         9|
|    1|         6|2018-01-06|   BGM| DTW| 6.909090909090909|        11|
|    1|         7|2018-01-07|   BGM| DTW|3.5384615384615383|        12|
|    1|         7|2018-01-07|   BGM| DTW|3.5384615384615383|        12|
|    1|         8|2018-01-08|   BGM| DTW| 4.866666666666666|        14|
|    1|         8|2018-01-08|   BGM| DTW| 4.866666666666666|        14|
|    1|         9|2018-01-09|   BGM| DTW| 3.235294117647059|        16|
|    1|         9|2018-01-09|   BGM| DTW| 3.235294117647059|        16|
|    1|        10|2018-01-10|   BGM| DTW|4.2631578947368425|        18|
|    1|        10|2018-01-10|   BGM| DTW|4.2631578947368425|        18|
|    1|        11|2018-01-11|   BGM| DTW|1.6666666666666667|        20|
+-----+----------+----------+------+----+------------------+----------+
only showing top 20 rows
```

Transformación JOIN

Método *join*: misma operación que en SQL con misma semántica.

- Si el nombre de columna es el mismo en ambos DF, se indica como string. Si no, se especifica una condición entre columnas que devuelva una columna de bool.
- Provoca movimiento de datos entre nodos y es la operación más pesada. Antes de usarla, debemos pensar si no hay más remedio. Muchas veces se puede sustituir por agregaciones con una ventana (sección siguiente).

Skip to main content

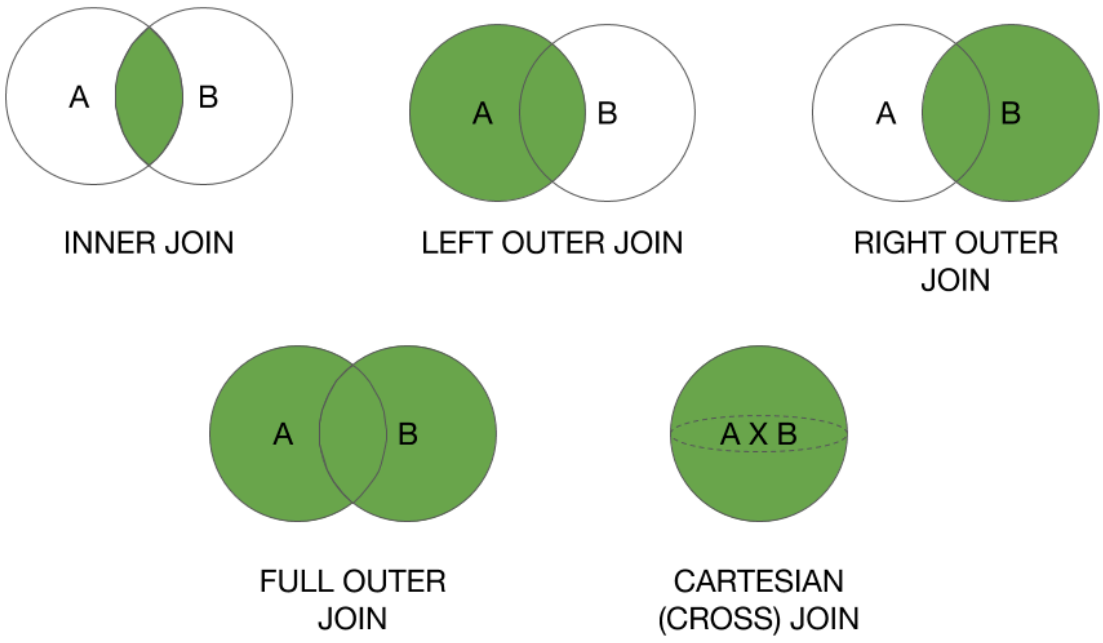


Fig. 6. Tipos de join.

Existen los mismos tipos de JOIN que en SQL: inner, cross, outer, full, full_outer, left, left_outer, right, right_outer, left_semi, left_anti, incluso por condición arbitraria. Si hay matching de una fila con varias filas del otro DF, se incrementa el número de filas del resultado respecto al número de filas original.

Ejemplos:

- Cuando las columnas por las que se hace el matching tienen el mismo nombre en ambos DF, se pueden especificar dichas columnas como una lista de strings.

```
tableA = spark.createDataFrame(
    [("a", 1, "010"), ("b", 4, "011"), ("r", 10, "012"), ("f", 13, "013")],
    ["a1", "a2", "f"]
)
tableA.show()

tableB = spark.createDataFrame(
    [("blue", 3, "023"), ("red", 10, "012"), ("orange", 3, "009"), ("yellow", 13, '
    ["b1", "b2", "f"]
)
tableB.show()

# el resultado de un left_outer tiene, como mínimo, tantas filas como el DF de la i
# si hay filas de un DF que tienen más de una coincidencia con el otro DF
joined_df = tableA.join(tableB, on = ["f"], how = "left_outer")
joined_df.show()
```

Skip to main content

```
+---+---+---+
| a1| a2|  f|
+---+---+---+
|  a|  1|010|
|  b|  4|011|
|  r| 10|012|
|  f| 13|013|
+---+---+---+
```

```
+------+---+---+
|    b1| b2|  f|
+------+---+---+
|  blue|  3|023|
|   red| 10|012|
|orange|  3|009|
|yellow| 13|013|
+------+---+---+
```

```
+---+---+---+------+----+
|  f| a1| a2|    b1|  b2|
+---+---+---+------+----+
|010|  a|  1|  NULL|NULL|
|011|  b|  4|  NULL|NULL|
|012|  r| 10|   red|  10|
|013|  f| 13|yellow|  13|
+---+---+---+------+----+
```

El tipo de JOIN por defecto es **inner** cuando no especificamos el argumento `how`: el resultado solo contiene filas que cumplen la condición en ambos DF. Si las columnas que se corresponden **no** tienen el mismo nombre:

```
tabla_izq = tableA.withColumnRenamed("f", "f1")
tabla_der = tableB.withColumnRenamed("f", "f2")

joined_df = tabla_izq.join(tabla_der, on = tabla_izq.f1 == tabla_der.f2) # inner pc
joined_df.show()
```

```
+---+---+---+------+---+---+
| a1| a2| f1|    b1| b2| f2|
+---+---+---+------+---+---+
|  r| 10|012|   red| 10|012|
|  f| 13|013|yellow| 13|013|
+---+---+---+------+---+---+
```

Skip to main content

> **Note**
>
> Cuando ya existen columnas con el mismo nombre en ambos DF y no van a ser utilizadas en la condición de join, o bien cuando sí van a utilizarse pero la condición incluye al mismo tiempo otras columnas con distintos nombres (y eso debemos expresar la condición mediante una operación con objetos columna), el DF resultante acabará teniendo columnas repetidas con el mismo nombre. En dicho caso, las columnas con el mismo nombre no podrán utilizarse después, ya que obtendremos un error de "referencia ambigua". Para ese caso, se suele utilizar la operación `alias` de los DF (no de los objetos columna). Con esto, podemos referirnos a las columnas de un DF o del otro aunque tengan el mismo nombre, simplemente utilizando `nombre_alias.nombre_columna` como si nombre de la columna.

```
error_df = tableA.join(tableB, on = (tableA.f == tableB.f) & (tableA.a2 == tableB.b
# error_df.select("f")  # error! AMBIGUOUS_REFERENCE: Reference 'f' is ambiguous

# Para evitar el error anterior, ponemos un alias a los DF
joined_df = tableA.alias("A").join(tableB.alias("B"), on = ["f"], how = "left_outer
joined_df.select("A.*", "b1", "b2").show()
```

```
+---+---+---+------+----+
|  f| a1| a2|    b1|  b2|
+---+---+---+------+----+
|010|  a|  1|  NULL|NULL|
|011|  b|  4|  NULL|NULL|
|012|  r| 10|   red|  10|
|013|  f| 13|yellow|  13|
+---+---+---+------+----+
```

Un tipo especial no contemplado en la figura es el *left_semi*: es una especie de *filter* sobre el DataFrame A donde la condición de filtrado es "tener matching con alguien de B". El resultado contiene solo las columnas de A (por eso se puede considerar como un mero filtrado). No se añade ninguna columna de B al resultado.

```
# el resultado contiene sólo las columnas de tableA, y solo las filas que tienen al
# un matching con filas de B
tableA.join(tableB, "f", "left_semi").show()
```

```
+---+---+---+
|  f| a1| a2|
```

Skip to main content

```
|012|  r| 10|
|013|  f| 13|
+---+---+---+
```

Existe un tipo análogo de *join* llamado *left_anti*, cuyo comportamiento es retener aquellas filas del DF izquierdo (**sin añadirles columnas adicionales**) que no tienen correspondencia con ninguna fila del DF derecho. *Left_anti* sería la operación complementaria de *left_semi* ya que el resultado contendría justamente las filas del DF de la izquierda que serían excluidas por *left_semi*.

```
# el resultado contiene sólo las columnas de tableA, y solo las filas
# que no tienen ningún matching con filas de B
tableA.join(tableB, ["f"], "left_anti").show()
```

```
+---+---+---+
|  f| a1| a2|
+---+---+---+
|010|  a|  1|
|011|  b|  4|
+---+---+---+
```

User-Defined Functions (UDF)

Se trata de un mecanismo para convertir una función convencional escrita en Python, R, Java o Scala en una función que puede ser llamada por los executors sobre cada fila de un DF de Spark, procesando los valores de una o varias columnas concretas.

Primero escribimos la función en Python que procesa un solo elemento, y luego la envolvemos en un objeto UDF que permite invocarla sobre una columna completa. Una vez hecho esto, cuando la utilicemos Spark automáticamente enviará el código de nuestra función a los executors para que la invoquen una vez por cada fila. Cada executor invoca a nuestra función sobre las porciones (particiones) de datos presentes en el executor. Sería el análogo a la función *map* que se suele usar con RDDs para transformar cada fila, pero aplicado a DataFrames.

Se utiliza habitualmente dentro de **withColumn()** para *añadir una nueva columna a un DF existente,* con el resultado de aplicarle a cada fila nuestra función, en la que intervienen solo ciertas columnas de la fila. A la hora de implementar la función de Python que después será envuelta por el objeto *udf*(), debemos pensar en una función que procesa valores individuales pasados como argumento, sin tener en cuenta que después esos valores

Skip to main content

UDF mediante una llamada a la función *udf*() de pyspark.sql.functions, y a partir de ese momento, podremos invocar dicho objeto sobre columnas completas (objetos Column) en lugar de sobre valores individuales.

Ejemplo:

```
from pyspark.sql import functions as F
from pyspark.sql import types as T

# Definimos una función de Python que multiplica un número por 1.6
def milesToKm(miles):
    """
    Función que traduce a km el número de millas que pasamos como argumento

    :param miles: distancia en millas
    :returns: distancia traducida a km
    """
    return(1.6 * miles) # multiplica un número por 1.6

# Ahora la envolvemos en un objeto UDF, especificando el tipo devuelto

toKmUDF = F.udf(milesToKm, T.DoubleType())  # indicamos el tipo de dato del valor d

# Finalmente, la utilizamos, pasándole una columna completa

result_df = flightsDF.withColumn("DistKm", toKmUDF(F.col("Distance")))
result_df.select("FlightDate", "Origin", "Dest", "Distance", "DistKm").distinct().s
```

```
+----------+------+----+--------+------+
|FlightDate|Origin|Dest|Distance|DistKm|
+----------+------+----+--------+------+
|2018-01-18|   DTW| BTV|   537.0| 859.2|
|2018-01-17|   DTW| SAT|  1214.0|1942.4|
|2018-01-04|   BQK| ATL|   238.0| 380.8|
|2018-01-05|   SYR| LGA|   198.0| 316.8|
|2018-01-31|   STL| CVG|   308.0| 492.8|
|2018-01-07|   CLT| JFK|   541.0| 865.6|
|2018-01-01|   FLL| PHL|   992.0|1587.2|
|2018-01-01|   MCO| PVD|  1072.0|1715.2|
|2018-01-01|   SAN| FLL|  2269.0|3630.4|
|2018-01-02|   BOS| MSY|  1368.0|2188.8|
+----------+------+----+--------+------+
only showing top 10 rows
```

Al invocar a la UDF (recordemos que *withColumn* es una transformación, por lo que sólo se ejecutará cuando encontremos una acción más adelante), Spark se ocupará de:

- Serializar el código Python de la función *milesToKm*.
- Enviarlo por la red a los workers en los que existan particiones del DF *flightsDF*.

Skip to main content

- Para cada fila de la partición, transformar el tipo de dato de la columna Distance, que será Integer de Java (Scala), al tipo entero de Python, y entonces invocar a *milesToKm* pasándole los valores convertidos a Python. Esta operación implica un coste computacional al convertir tipos de un lenguaje (Java) a otro (Python) pero es inevitable en las UDF de Python, salvo en la versión 2.4 en los casos que puedan solucionarlo las PandasUDF[^7].

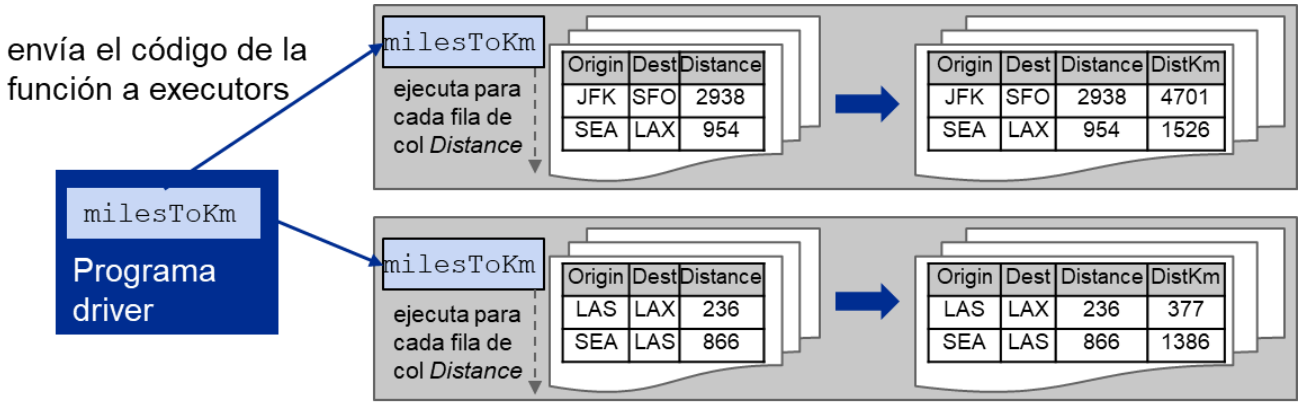


Fig. 7. Ejecución en los executors de la UDF DistKm que envuelve a milesToKm

En el ejemplo anterior, la UDF es muy sencilla y se limita a multiplicar todos los valores por 1.6. En casos donde la transformación sea simple, debemos primero tratar de implementarla combinando **funciones ya existentes de pyspark.sql.functions**. En este caso podríamos simplemente haber hecho

```
flightsDF.withColumn("DistKm", 1.6 * F.col("Distance"))
```

puesto que se puede hacer aritmética de columnas, y el operador producto existe para columnas y una constante. La mayoría de las veces, incluso en transformaciones complejas, es posible expresarlas utilizando combinaciones de funciones de Spark, y esa implementación **siempre será más eficiente que una UDF**, puesto que una UDF es una caja negra a efectos del optimizador Catalyst y no puede ser optimizada, mientras que las funciones de Spark sí están sujetas a optimizaciones incluso combinadas entre sí.

Ejecución de código SQL puro

Además de la API estructurada de Spark SQL, es posible expresar transformaciones sobre DataFrames utilizando lenguaje SQL puro. Para ello, primero hay que registrar el DataFrame como tabla de Hive. Generalmente se registra como vista temporal volátil para esta sesión, lo cual no crea ningún archivo en almacenamiento físico sino que solo añade metadatos

Skip to main content

argumento un string con la consulta SQL completa, que puede ser tan compleja como necesitemos. Spark automáticamente es capaz de analizarla y traducirla a trabajos de Spark que se ejecutan de forma distribuida y paralela en el cluster sobre nuestro DF.

```
# Registramos nuestro DF como una table temporal llamada flights

flightsDF.createOrReplaceTempView("flights")
result_df = spark.sql("select max(ArrDelay) from flights group by Origin")
result_df.show(10)
```

```
+-------------+
|max(ArrDelay)|
+-------------+
|       1209.0|
|        292.0|
|       1004.0|
|       1197.0|
|        607.0|
|       1432.0|
|       1205.0|
|       1091.0|
|       1249.0|
|         59.0|
+-------------+
only showing top 10 rows
```

Podemos salvar un DataFrame como una tabla persistente de Hive con el siguiente comando. En ese caso se guardará en el almacenamiento que Hive tenga configurado (suele ser HDFS por defecto) y en la ruta que tenga configurado (alguna ubicación concreta de HDFS, generalmente colgante de /hive).

`flightsDF.write.saveAsTable("flightsTable")`

API de lectura y escritura de DataFrames

La lectura y escritura de DataFrames de todo tipo de fuentes de datos se realiza mediante una API uniforme: es otra de las **grandes fortalezas** de Spark, ya que existen **conectores** para casi cualquier fuente de datos:

- Ficheros almacenados en Amazon S3, Google Cloud Storage, HDFS ...
- Formatos de archivo como TXT, CSV, JSON, XML, libsvm
- Bases de datos relacionales, para las que se usa el protocolo JDBC habitual (MySQL, PostgreSQL, Oracle, Microsoft SQL server...)

Skip to main content

- Conector para el índice inverso de ElasticSearch, ...
- Fuentes de datos en streaming, por ejemplo una cola de Kafka

El objeto `spark.read` es un objeto de la clase `DataFrameReader` y realiza las lecturas. Podemos configurar varias opciones mediante `option("key", "value")`. Existe una opción genérica `format("formato")` donde "formato" es "csv", "jdbc", "mongo", "parquet", "json" ... y termina con el método `load("/path/to/file")`. Los formatos frecuentes tienen un atajo para no usar load, ej: `csv("/path/to/file.csv")`.

La lectura **siempre devuelve un DataFrame**, independientemente de la fuente de datos, lo cual facilita mucho su uso. Entre otras opciones, podemos pasarle el esquema explícito en el momento de hacer la lectura, con la opción .schema(myschema). Para ficheros CSV podemos indicar qué hacer con las filas mal formadas (descartarlas, fallar, o devolverlas a null), y también cuál es el carácter delimitador que separa los campos mediante `.option("delimiter", "\|")` donde estamos indicando en este caso que es `|`.

En archivos en formato Parquet y ORC, el tipo de dato de cada columna se almacena en el propio fichero, por lo que los tipos leídos siempre son correctos y coinciden exactamente con los que se guardaron. En otros casos, Spark puede intentar inferir el tipo de dato de cada columna, aunque se le puede indicar que no lo haga (y en ese caso leerá todo como string), o darle el esquema. Los CSV son más problemáticos. Spark no admite separador de más de un carácter, y además no permite guardar tipos compuestos (columnas cuyo tipo de dato es una tupla, o es una lista), mientras que Parquet sí.

Formato general:

```
myDF = spark.read.format("format")\
            .option("key","value")\
            .schema(mySchema)\
            .load("/path/to/file")
```

Ejemplos:

```
flightsDF = spark.read.format("csv")\
                .option("mode", "FAILFAST")\
                .option("header", "true")\
                .option("inferSchema", "true")\
                .option("path","flights.csv")\
                .load()
```

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
```

Skip to main content

```
                            StructField("Origin", StringType(), nullable = True),
                            StructField("ArrDelay", IntegerType(), nullable = True),
                            #  resto de campos del esquema
                           ])

flightsDF = spark.read.schema(flightsSchema)\
                      .option("mode", "DROPMALFORMED")\\
                      .csv("flights.csv")
```

Otro ejemplo: lectura de una base de datos MongoDB (ver detales en https://www.mongodb.com/docs/spark-connector/current/batch-mode/batch-read-config)

```
collection_df = spark.read.format("mongodb")\
                          .option("connection.uri", "mongodb://localhost:27017")\
                          .option("collection", "collection_name")
```

La escritura de un DF la realiza el objeto **myDF.write**, de la clase DataFrameWriter.

Se le pasan opciones mediante .option("key", "value"), y la opción .format("formato") donde casi todos los formatos frecuentes tienen ya un atajo para no tener que usarlo. Termina con el método **save**("/path/to/file") que, cuando es HDFS, **crea un nuevo directorio**. En ese directorio **se crean tantos ficheros como particiones tenga el DataFrame**. Algunas opciones interesantes:

- **mode**("modo"): modo es "overwrite" ó "append" ó "error" (por defecto), ó "ignore"
- **partitionBy**() cuando guardamos en formato Parquet, crea una estructura de directorios en base a los valores de una variable categórica que acelera operaciones de filtrado cuando leamos de nuevo el fichero.
- **saveAsTable**() en lugar de save() al final, para guardar el DF físicamente en el formato y ubicación que tenga configurado Hive (en el caso de Databricks se guarda en formato Delta sobre DBFS), y añadir los metadatos correspondientes al metastore de Hive.

Ejemplos:

```
flightsWeekendDF.write.partitionBy("Origin")\
                      .format("parquet")\
                      .save("/path/new/folder")
```

La parte .format().save() se puede sustituir por .parquet("/path/folder"). En ese directorio Spark creará subcarpetas llamadas "Origin=JFK", "Origin=LAX", etc y dentro de cada subcarpeta se guardarán los datos de esa partición, creando dentro tantos ficheros como particiones físicas tenga el DF que estamos salvando

Skip to main content

Otra opción es salvar físicamente los datos en una tabla del metastore de Hive, en el formato que venga configurado por defecto y en la ubicación que venga configurada por defecto en Spark. Habitualmente el formato de fichero es Parquet, aunque si ejecutamos en la plataforma de Databricks, el formato será Delta. La ruta depende de lo que indique la propiedad `spark.sql.warehouse.dir` y suele tener el valor `/user/hive/warehouse` o `/hive/warehouse/` o algo similar.

```
# Salvamos físicamente los datos en una tabla Hive en Parquet
flightsWeekendDF.write.**saveAsTable**("flightsWeekends")
```

Lectura y escritura de BBDD relacionales

Spark puede leer de cualquier BBDD relacional, como MySQL, PostgreSQL, Oracle, SQL Server...mediante el protocolo habitual **JDBC.** Spark hace **predicate pushdown**: una operación de filtrado de Spark puede traducirla automáticamente en un filtrado nativo del datasource. Hay reglas de cuándo será capaz y cuándo no. También es posible hacerlo con archivos Parquet.

Cuando leemos con Spark desde una BBDD relacional, los executors de Spark se conectan en paralelo a la base de datos para leer simultáneamente porciones distintas de la misma tabla al mismo tiempo.

Federación de datos con Spark

Al igual que se puede construir una base de datos federada como una base de datos virtual formada por BBDD heterogéneas, de forma transparente al usuario, es posible usar Spark para construir DataFrames a partir de fuentes de datos diversas, y después manipularlos y cruzarlos mediante código Spark, sin importar su origen. Spark actúa como una **capa de abstracción común** y lleva a cabo las operaciones que no puedan ser traducidas a consultas nativas

Skip to main content



Jobs, stages y tasks (trabajos, etapas y tareas)

- Un **job** (trabajo) de Spark es todo el procesamiento necesario *para llevar a cabo una acción del usuario*

Por ejemplo: df.count(), df.take(4), df.show(), df.read(...), df.write(...), etc). Cada *job* se divide en una serie de *stages* (etapas).

- Un **stage** (etapa) es todo el procesamiento que puede llevarse a cabo *sin mover datos entre nodos*. Cada nodo hace exactamente el mismo procesamiento aplicado a diferentes particiones del mismo DataFrame que están procesando todos.

Cuando hay una operación que implica movimiento de datos (*shuffle*), finaliza un stage y se crea otro nuevo. Ej: df.join(...), df.groupBy(...).agg(...)

- Una **task** (tarea) de Spark es el procesamiento aplicado por un core físico (CPU) a una partición de un RDD. La tarea es la unidad mínima de trabajo de Spark.

Skip to main content

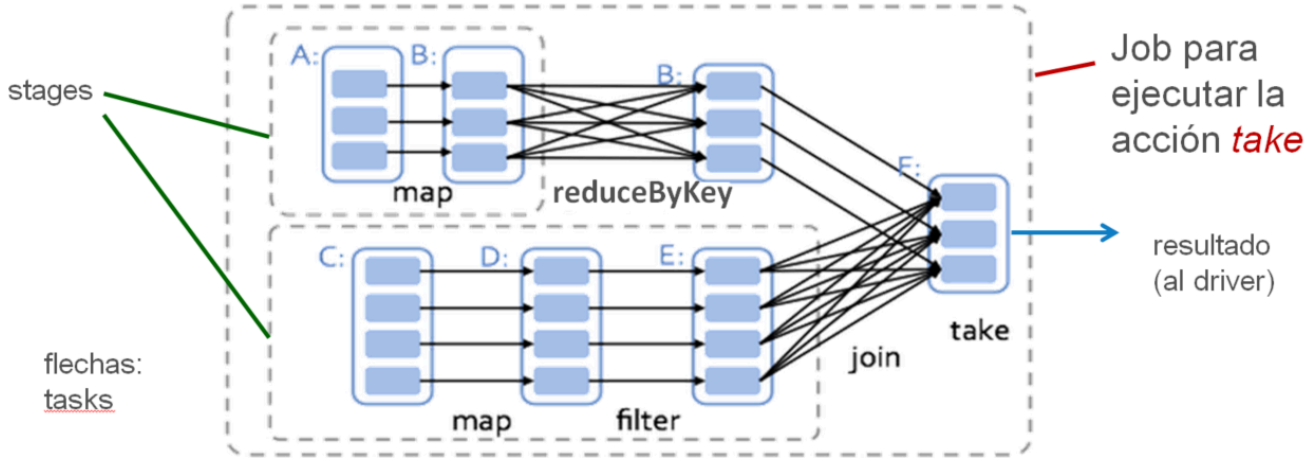


Fig. 9. Representación de Jobs, stages y tasks en Spark

- Bill Chambers y Matei Zaharia. *Spark: The Definitive Guide* O'Reilly, 2018.
- La web oficial de Spark, https://spark.apache.org/docs/latest/ contiene una extensa documentación.
- Más detalles sobre jobs, staes y tasks en la doc. oficial: https://spark.apache.org/docs/latest/job-scheduling.html
- RDD programming guide: http://spark.apache.org/docs/latest/rdd-programming-guide.html
- Spark SQL programming guide: http://spark.apache.org/docs/latest/sql-programming-guide.html
- Fuente: Apache Spark Analytics Made Simple – Highlights from the Databricks Blog
- Pandas-on-Spark (pyspark-pandas): http://spark.apache.org/docs/latest/sql-pyspark-pandas-with-arrow.html